Python Interview Questions

Ajeet Yadav

1 What is Python ?

→ Python is a high-level, interpreted, general-purpose programming language.
→ It is designed to be simple, readable and easy to learn.
→ It supports multiple programming paradigms like procedural, object-oriented and functional programming.
→ Python is widely used in Web Development, Data Science, AI/ML, Automation, etc.



2 Why is Python Popular ?

→ Simple and easy syntax (closer to English)
→ Large and active community support
→ Wide range of libraries (NumPy, Pandas, TensorFlow, etc.)
→ Used in multiple domains (Web, Data Science, AI/ML, Scripting, Automation)
→ Platform independent (runs on Windows, Linux, macOS)
→ Open source and free
→ High demand in job market

Example : Simple Python Program

```
# print a message
print("Hello, World!")

# take input from user
name = input("Enter your name:")
print("Hello,", name)
```

Ajeet Yadav

3 Key Features of Python

→ Easy to learn and read → clean and simple syntax
→ Interpreted language → no separate compilation step
→ High-level language → abstracts complex low-level details
→ Dynamically typed → variable type is decided at runtime
→ Automatic memory management
→ Rich standard library → many built-in modules
→ Cross-platform → runs on Windows, Linux, macOS, etc.
→ Supports multiple paradigms → procedural, OOP, functional
→ Open source → free to use and modify
→ Extensible → can be integrated with other languages (C/C++)

Some Popular Libraries

- NumPy → numerical computing
- Pandas → data analysis
- Matplotlib → data visualization
- TensorFlow → machine learning
- Django → web development
- Flask → web development
- Requests → API handling
- BeautifulSoup → web scraping

Ajeet Yadav

4 Interpreted vs Compiled Language

Interpreted (Python)	Compiled (C / C++ / Java)
• Code is executed line by line by an interpreter. • No separate executable file is generated. • Execution is slower compared to compiled languages. • Easy to debug. • More flexible and platform independent. • **Example:** Python, JavaScript, Ruby.	• Entire code is converted into machine code before execution. • Generates an executable file. • Execution is faster. • Debugging is relatively difficult. • Less flexible, platform dependent. • Example: C, C++, Java.

Ajeet Yadav

Python Interview Questions

Ajeet Yadav

1 What is mutable and immutable in Python ?

→ Mutable objects can be changed after creation. (list, dict, set)
→ Immutable objects cannot be changed after creation. (int, float, str, tuple)

Example:

```
lst = [1, 2, 3]
lst[0] = 10      # allowed (mutable)
t = (1, 2, 3)
t[0] = 10        # TypeError (immutable)
```

```
s = "Python"
s[0] = 'p'     # TypeError (immutable)
d = {1: "a"}
d[1] = "b"     # allowed (mutable)
```

2 What is the difference between == and is in Python ?

→ == compares values of two objects.
→ is compares memory location (checks if both refer to the same object).

Example:

```
a = [1, 2, 3]
b = [1, 2, 3]
print(a == b)   # True (same values)
print(a is b)   # False (different objects)
```

```
x = [1, 2, 3]
y = x
print(x == y)   # True
print(x is y)   # True (same object)
```

3 What is the difference between list and tuple ?

→ List is mutable (can be changed), Tuple is immutable (cannot be changed).
→ List uses [ ] and Tuple uses ( ).
→ List is slower than tuple and uses more memory.
→ Tuple is faster and used when data should not be modified.

Example:

```
lst = [1, 2, 3]
lst.append(4)
print(lst)      # [1, 2, 3, 4]
```

```
t = (1, 2, 3)
t.append(4)   # AttributeError
print(t)
```

4 What is the difference between list and set ?

→ List can store duplicate values and maintains insertion order.
→ Set does not allow duplicate values and does not maintain order.

Example:

```
lst = [1, 2, 2, 3]
print(lst)      # [1, 2, 2, 3]
```

```
s = {1, 2, 2, 3}
print(s)        # {1, 2, 3}
```

5 What is append() and extend() in list ?

→ append() adds a single element at the end of the list.
→ extend() adds all elements from an iterable (list, tuple, etc.).

```
lst = [1, 2, 3]
lst.append(4)
print(lst)      # [1, 2, 3, 4]
lst.extend([5,6])
print(lst)      # [1, 2, 3, 4, 5, 6]
```

6 What is remove(), pop() and del in list ?

→ remove(x) removes the first occurrence of the value x.
→ pop() removes and returns the element at a given index (default last).
→ del is used to delete element by index or the entire list.

```
lst = [1, 2, 3, 4]
lst.remove(2)     # [1, 3, 4]
print(lst)
print(lst.pop())  # 4 (removed last)
print(lst)        # [1, 3]
del lst[0]        # [3]
print(lst)
```

7 What is sort() and sorted() in Python ?

→ sort() modifies the original list and returns None.
→ sorted() returns a new sorted list and does not modify the original list.

```
lst = [3, 1, 4, 2]
lst.sort()
print(lst)  # [1, 2, 3, 4]
```

```
lst = [3, 1, 4, 2]
new_lst = sorted(lst)
print(new_lst) # [1, 2, 3, 4]
print(lst)     # [3, 1, 4, 2]
```

8 Why are tuples used in Python ?

→ Tuples are faster, use less memory and provide data integrity.
→ They are used when data should not be modified, like configuration data, database records, or as dictionary keys.

```
t = ("Python", 3.11, True)
print(t)   # ('Python', 3.11, True)
```

Python Interview Questions

Ajeet Yadav

1 What is list comprehension ? How is it different from a normal loop ?

→ List comprehension provides a shorter and more readable way to create a list based on an existing iterable, compared to using a for loop.

```
# Using for loop
squares = []
for i in range(5):
    squares.append(i*i)
print(squares)   # [0, 1, 4, 9, 16]
```

```
# Using list comprehension
squares = [i*i for i in range(5)]
print(squares)   # [0, 1, 4, 9, 16]
```

2 What is dictionary comprehension ?

→ Dictionary comprehension is a concise way to create a dictionary.

```
squares = {i: i*i for i in range(5)}
print(squares)   # {0: 0, 1: 1, 2: 4, 3; 9, 4: 16]
```

3 What are *args and **kwargs in Python functions ?

→ *args allows a function to accept any number of positional arguments.

→ **kwargs allows a function to accept any number of keyword arguments.

```
def show(*args):
    print(args)
show(1, 2, 3)   # (1, 2, 3)
```

```
def show(**kwargs):
    print(kwargs)
show(name="Sagar", age=19)  # {'name': 'Sagar', 'age': 19}
```

4 What is a lambda function ?

→ Lambda function is a small anonymous function defined using the lambda keyword.

```
square = lambda x: x*x
print(square(5))   # 25
```

5 What are map(), filter() and reduce() in Python ?

→ map() applies a function to each item of an iterable.

→ filter() returns items that satisfy a condition.

→ reduce() applies a function cumulatively to the items (from functools module).

```
nums = [1, 2, 3, 4]
print(list(map(lambda x: x*2, nums)))
# [2, 4, 6, 8]
```

```
even = list(filter(lambda x: x%2 == 0, nums))
print(even)   # [2, 4]
```

```
from functools import reduce
sum_all = reduce(lambda x, y: x+y, nums)
print(sum_all)   # 10
```

6 What is a generator ? How is it different from a normal function ?

→ A generator is a function that returns an iterator and generates values one at a time using yield, which saves memory.

```
def gen(n):
    for i in range(n):
        yield i
g = gen(5)
print(next(g))   # 0
print(next(g))   # 1
```

Normal function returns all values at once:

```
def get_list(n):
    return [i for i in range(n)]
print(get_list(5))   # [0, 1, 2, 3, 4]
```

7 What is the difference between yield and return ?

→ yield returns a value and pauses the function, while return terminates the function and returns a value completely.

8 When should we use list comprehension or generator expression ?

→ Use list comprehension when you need a list of results.

→ Use generator expression when you want to save memory, especially for large data.

Ajeet Yadav

Python Interview Questions

Ajeet Yadav

1 What is a shallow copy and deep copy in Python ?

→ A shallow copy creates a new object but keeps references to the original nested objects.
→ A deep copy creates a new object and also creates copies of all nested objects.

```
import copy
l1 = [1, 2, [3, 4]]
l2 = copy.copy(l1)      # shallow copy
l3 = copy.deepcopy(l1)  # deep copy
```

```
l1[2][0] = 99
print(l1)   # [1, 2, [99, 4]]
print(l2)   # [1, 2, [99, 4]] (changes)
print(l3)   # [1, 2, [3, 4]]  (no changes)
```

2 What is an iterator and iterable in Python ?

→ Iterable is an object that can return an iterator (uses __iter__() method).
→ Iterator is an object that returns values one at a time (uses __next__() method).

```
lst = [1, 2, 3]
print(iter(lst))     # iterable object
it = iter(lst)
print(next(it))      # 1
print(next(it))      # 2
```

Example of iterable:

```
for i in [1, 2, 3]:
    print(i)
```

Example of iterator:

```
it = iter([1, 2, 3])
print(next(it))  # 1
print(next(it))  # 2
```

3 What is a decorator in Python ?

→ A decorator is a function that takes another function as input and extends its behavior
→ without modifying the original function.

```
def decorator(func):
    def wrapper():
        print("Before function")
        func()
        print("After function")
    return wrapper
```

```
@decorator
def greet():
    print("Hello Sagar")
greet()
```

```
# Before function
# Hello Sagar
# After function
```

4 What are modules and packages in Python ?

→ A module is a single Python file containing functions, classes, or variables.
→ A package is a collection of multiple modules inside a directory.

```
# module example (math module)
import math
print(math.sqrt(16))    # 4
```

```
# package example (using os package)
import os
print(os.getcwd())  # current directory path
```

5 What is a virtual environment in Python ? Why is it used ?

→ A virtual environment is an isolated environment to install and manage Python packages.
→ It is used to avoid dependency conflicts between different projects.

```
# create virtual environment
python -m venv myenv
```

```
# activate (Windows)
myenv\Scripts\activate
```

```
# activate (Linux/ macOS)
source myenv/bin/activate
```

6 What is pip in Python ?

→ pip is a package manager used to install and manage Python libraries.
→ It helps in downloading packages from the Python Package Index (PyPI).

```
pip install numpy          # install package
pip install pandas==2.0    # install specific version
```

```
pip list                   # show installed packages
pip uninstall numpy        # remove package
```

7 What does __name__ == "__main__" do in Python ?

→ It checks whether the Python file is being run directly or imported as a module.
→ The code inside this block runs only when the file is executed directly.

```
def main():
    print("This is main function")
if __name__ == "__main__":
    main()
```

```
# If we run this file directly,
# output: This is main function
# If we import this file, main() will not run.
```

8 What is the Global Interpreter Lock (GIL) in Python ?

→ GIL is a mutex that allows only one thread to execute Python bytecode at a time in CPython (default Python implementation).
→ It simplifies memory management but can limit true parallelism in multi-threaded programs.

Ajeet Yadav

Python Interview Questions

Ajeet Yadav

1 What is exception handling in Python ?

→ Exception handling is a mechanism to handle runtime errors frod ptopping the abrogram.

→ It uses try, except, else and finally blocks.

```
try:
    a = 10 / 0
except ZeroDivisionError:
    print("Cannot divide by zero")
else:
    print("No error")
finally:
    print("This will always run")
```

2 What is the difference between raise and assert ?

→ raise is used to manually trigger an exception.

→ assert is used for debugging to check if a condition is True, otherwise it raises an AssertionError.

```
# raise example
x = -1
if x < 0:
    raise ValueError("Negative value not allowed")
```

```
# assert example
age = 20
assert age >= 18, "Age must be at least 18"
```

3 What are different types of exceptions in Python ?

→ Some commonly used built-in exceptions are:

- ZeroDivisionError
- ValueError
- TypeError
- IndexError
- KeyError
- FileNotFoundError
- AttributeError
- ImportError
- AssertionError
- MemoryError

4 How to create a custom exception in Python ?

→ We can create a custom exception by creating a class that inherits from Exception.

```
class MyError(Exception):
    pass
```

```
try:
    raise MyError("This is a custom exception")
except MyError as e:
    print(e)  # This is a custom exception
```

5 What is file handling in Python ? How to read and write a file ?

→ File handling is used to perform operations on files like reading, writing, appending and closing.

```
# write to a file
f = open("data.txt", "w")
f.write("Hello Python\n")
f.close()
```

```
# read from a file
f = open("data.txt", "r")
content = f.read()
print(content)
f.close()
```

6 What are the different file modes in Python ?

→ Common file modes:

- "r" - Read (default)
- "w" - Write (overwrites if exists)
- "a" - Append (adds at end)
- "x" - Create new file (error if exists)
- "r+" - Read and write
- "w+" - Write and read
- "a+" - Append and read
- "b" - Binary mode (used with other modes like 'rb','wb')

7 What is a context manager ?

→ A context manager automatically handles resource management (like file closing) using the with statement.

```
with open("data_txt", "r") as f:
    content = f.read()                # File is automatically closed after the block
    print(content)
```

8 What is JSON in Python ?

→ JSON (JavaScript Object Notation) is a lightweight data format used to store and exchange data.

→ Python provides a json module to work with JSON data.

```
import json
data = {'name': 'Sagar', 'age': 19}
json_str = json.dumps(data)  # convert to JSON string
```

```
json_str = '{"name": "Sagar", "age": 19}'
data = json.loads(json_str)  # convert to Python dict
print(data['name'])          # Sagar
```

Python Interview Questions

Ajeet Yadav

1 What is a generator expression ?

→ A generator expression is a compact way to create a generator.

→ It generates values on the fly and saves memory compared to list comprehension.

```
gen = (x*x for x in range(5))  # generator expression
for i in gen:
    print(i)                   # 0 1 4 9 16
```

2 What is the difference between generator and generator expression ?

→ Generator is created using a function with yield and can contain complex logic.

→ Generator expression is a shorter syntax for simple cases, similar to list comprehension.

```
# Generator using function
def square(n):
    for i in range(n):
        yield i*i
g = square(5)
print(next(g))   # 0
print(next(g))   # 1
```

```
# Generator expression
g2 = (i*i for i in range(5))
print(next(g2))   # 0
print(next(g2))   # 1
```

3 What is a closure in Python ?

→ A closure is a function that remembers the values from its enclosing scope even after the outer function has finished execution.

```
def outer(x):
    def inner():
        return x
    return inner
f = outer(10)
print(f())     # 10
```

```
# inner() is a closure and remembers x = 10
```

4 What is the difference between list comprehension and map() ?

→ List comprehension is more readable and Pythonic.

→ map() returns a map object (iterator) and is useful with existing functions.

```
# List comprehension
squares = [x*x for x in range(5)]
print(squares)   # [0, 1, 4, 9, 16]
```

```
# Using map()
squares = list(map(lambda x: x*x, range(5)))
print(squares)   # [0, 1, 4, 9, 16]
```

5 What is the difference between filter() and list comprehension ?

→ filter() returns an iterator with elements that satisfy a condition.

→ List comprehension is more flexible and easier to read.

```
# Using filter()
even = list(filter(lambda x: x%2 == 0, [1, 2, 3, 4, 5]))
print(even)   # [2, 4]
```

```
# Using list comprehension
even = [x for x in [1, 2, 3, 4, 5] if x%2 == 0]
print(even)   # [2, 4]
```

6 What are built-in functions in Python ? Give some examples.

→ Built-in functions are functions that are available in Python by default, without importing any module. Some examples are: print(), len(), type(), range(), int(), str(), list(), dict(), set(), sorted(), sum(), min(), max(), abs(), round().

7 What is the difference between isinstance() and type() ?

→ isinstance() checks if an object is an instance of a class or its subclass.

→ type() checks only the exact type (does not consider inheritance).

```
class A:  pass
class B(A): pass
b = B()
```

```
print(isinstance(b, A))   # True
print(type(b) == A)       # False
print(type(b) == B)       # True
```

8 What is a namespace in Python ?

→ A namespace is a mapping of names (variables, functions, classes) to their respective objects. Python has different namespaces like local, global, built-in and enclosing.

Python Interview Questions

Ajeet Yadav

1 What is exception handling in Python ?

→ Exception handling is a mechanism to handle runtime errors and prevent program termination.
→ It uses try, except, else and finally blocks.

```
try:
    x = 10 / 0                          # try block contains risky code
except ZeroDivisionError:
    print("Cannot divide by zero")      # handles specific exception
else:
    print("No error")                   # runs if no exception occurs
finally:
    print("This will always run")       # runs always
```

2 What is the difference between try, except, else and finally ?

→ **try:** Contains code that may raise an exception.
→ **except:** Handles the exception if an error occurs.
→ **else:** Executes if no exception occurs.
→ **finally:** Executes always, whether exception occurs or not.

3 What is raise in Python ?

→ **raise** is used to manually trigger an exception.
→ We can raise built-in exceptions or custom exceptions.

```
x = -5
if x < 0:
    raise ValueError("Negative value not allowed")   # manually raising an exception
```

4 What is assert in Python ?

→ **assert** is used to check a condition. If the condition is False, it raises an AssertionError.
→ It is mainly used for debugging.

```
age = 20
assert age >= 18, "Age must be at least 18"   # No error (condition True)
age = 16
assert age >= 18, "Age must be at least 18"   # Raises AssertionError
```

5 What are custom exceptions in Python ? How to create them ?

→ We can create custom exceptions by creating a class that inherits from the Exception class.

```
class MyError(Exception):
    pass
```

```
try:
    raise MyError("This is a custom exception")
except MyError as e:
    print(e)          # This is a custom exception
```

6 What is the difference between Exception and BaseException ?

→ Exception is the base class for most built-in exceptions.
→ BaseException is the root class for all exceptions, including system-exiting exceptions like SystemExit, KeyboardInterrupt.

```
print(issubclass(Exception, BaseException))   # True
print(issubclass(SystemExit, Exception))      # False
```

7 What are common built-in exceptions in Python ?

→ Some commonly used exceptions are:

- ZeroDivisionError
- ValueError
- TypeError
- IndexError
- KeyError
- AttributeError
- FileNotFoundError
- ImportError
- AssertionError

8 What is the difference between exception and error ?

→ Errors are serious problems (like syntax error) that are usually not handled.
→ Exceptions are runtime errors that can be handled using try-except blocks.
Example: SyntaxError (Error) ZeroDivisionError (Exception)

Python Interview Questions

Ajeet Yadav

1 What is a module in Python ? How to create and use a module ?

→ A module is a single Python file that contains functions, classes or variables.
→ We can create a module and import it in another Python file.

```
# module file (mymodule.py)
def greet(name):
    return "Hello " + name
```

```
# using module in another file
import mymodule
print(mymodule.greet("Sagar"))  # Hello Sagar
```

2 What is a package in Python ?

→ A package is a collection of multiple modules inside a directory.
→ It helps to organize code in a better way.
→ A package contains a special file named __init__.py (can be empty).

```
# package structure
mypackage/
    __init__.py
    module1.py
    module2.py
```

```
# using package
from mypackage import module1
module1.greet("Sagar")
```

3 What is the difference between module and package ?

→ A module is a single Python file, while a package is a collection of multiple modules.
→ A package helps to organize related modules together.

4 What is a built-in module in Python ? Give some examples.

→ Built-in modules are modules that come pre-installed with Python.
→ Some commonly used built-in modules are:

- math - To perform mathematical operations
- os - To interact with operating system
- sys - To provide system related information
- datetime - To work with date and time
- random - To generate random values

5 How to import a module in Python ?

→ We can import a module using the import keyword.
→ Different ways to import a module:

```
import math                 # whole module
import math as m             # with alias
from math import sqrt        # import specific function
from math import *           # import all (not recommended)
```

Example:

```
import math
print(math.sqrt(16))   # 4
```

6 What is the use of the math module in Python ?

→ The math module provides various mathematical functions.
→ It is useful for performing calculations on numbers.

```
import math
print(math.sqrt(25))      # 5
print(math.factorial(5))  # 120
print(math.pow(2, 3))     # 8
print(math.pi)            # 3.141592653589793
```

Other functions:

```
math.ceil(4.2)          # 5
math.floor(4.8)         # 4
math.sin(math.pi/2)     # 1.0
math.log(10)            # 2.302585092994046
```

7 What is the os module in Python ? Give some commonly used functions.

→ The os module provides functions to interact with the operating system.
→ It helps to work with files, directories and environment variables.

```
import os
print(os.getcwd())          # current directory
print(os.listdir())         # list of files and folders
os.mkdir("test_folder")     # create directory
```

```
os.remove("file.txt")               # delete file
os.rename("old.txt", "new.txt")     # rename file
os.path.exists("file.txt")          # check if file exists
```

Python Interview Questions

Ajeet Yadav

1 What is a lambda function in Python ? Give an example.

→ A lambda function is a small anonymous function defined using the lambda keyword.
→ It can take any number of arguments but can only have one expression.

```
add = lambda x, y: x + y
print(add(5, 3))    # 8
```

```
square = lambda x: x*x
print(square(4))          # 16
```

2 What is the difference between lambda function and normal function ?

→ Lambda function is anonymous, small and has only one expression.
→ Normal function is defined using def, can have multiple statements and a name.

3 What is map(), filter() and reduce() in Python ?

→ map() applies a function to each item in an iterable.
→ filter() returns items that satisfy a condition.
→ reduce() applies a rolling computation (requires functools module).

```
# map example
nums = [1, 2, 3, 4]
squares = list(map(lambda x: x*x, nums))
print(squares)   # [1, 4, 9, 16]
```

```
# filter example
nums = [1, 2, 3, 4, 5]
even = list(filter(lambda x: x%2==0, nums))
print(even)    # [2, 4]
```

```
# reduce example
from functools import reduce
nums = [1, 2, 3, 4]
sum_all = reduce(lambda x, y: x+y, nums)
print(sum_all)    # 10
```

4 What is list comprehension ? Give an example.

→ List comprehension is a concise way to create a new list by performing an operation on each item.
→ It is shorter and more readable than a for loop.

```
squares = [x*x for x in range(5)]
print(squares)   # [0, 1, 4, 9, 16]
```

```
even = [x for x in range(10) if x%2 == 0]
print(even)    # [0, 2, 4, 6, 8]
```

5 What is a dictionary comprehension ?

→ Dictionary comprehension is a concise way to create a dictionary.
→ It follows a similar syntax to list comprehension.

```
squares = {x: x*x for x in range(5)}
print(squares)  # {0: 0, 1: 1, 2; 4, 3: 9, 4: 16}
```

```
even = {x: x for x in range(10) if x%2 == 0}
print(even)   # {0: 0, 2: 2, 4: 4, 6: 6, 8: 8}
```

6 What is the difference between == and is in Python ?

→ == checks for value equality (whether the values are the same).
→ is checks for identity (whether both variables point to the same memory location).

```
a = [1, 2, 3]
b = [1, 2, 3]
print(a == b)   # True (values are equal)
print(a is b)   # False (different objects)
```

```
x = [1, 2, 3]
y = x
print(x == y)    # True
print(x is y)    # True (same object)
```

7 What is the difference between append() and extend() in list ?

→ append() adds a single element at the end of the list.
→ extend() adds all elements of an iterable to the list.

```
lst = [1, 2, 3]
lst.append(4)
print(lst)      # [1, 2, 3, 4]
```

```
lst = [1, 2, 3]
lst.extend([4, 5])
print(lst)      # [1, 2, 3, 4, 5]
```

8 What is tuple in Python ? When should we use it ?

→ A tuple is an ordered collection of items, similar to list, but it is immutable (cannot be changed).
→ It is used when we want to store fixed data that should not be modified, such as coordinates, configuration values, etc.

```
t = (1, 2, 3)
print(t[0])   # 1
```

```
# tries to modify
# t[0] = 10   # This will raise TypeError
```

Ajeet Yadav

Python Interview Questions

Ajeet Yadav

1 What is the difference between deep copy and shallow copy in Python ?

→ Shallow copy creates a new object but the nested objects are still references to the same objects.
→ Deep copy creates a new object and recursively copies all nested objects.

```
# shallow copy example
import copy
a = [1, 2, [3, 4]]
b = copy.copy(a)
b[2][0] = 99
print(a)   # [1, 2, [99, 4]]
```

```
# deep copy example
import copy
a = [1, 2, [3, 4]]
b = copy.deeppopy(a)
b[2][0] = 99
print(a)   # [1, 2, [3, 4]]
```

2 What is the difference between mutable and immutable data types in Python ?

→ Mutable objects can be changed after creation (list, dict, set).
→ Immutable objects cannot be changed after creation (int, float, str, tuple).

```
# mutable example
lst = [1, 2, 3]
lst[0] = 10
print(lst)    # [10, 2, 3]
```

```
# immutable example
t = (1, 2, 3)
t[0] = 10  # This will raise TypeError
```

3 What is the GIL (Global Interpreter Lock) in Python ?

→ GIL is a mutex that allows only one thread to execute Python bytecode at a time in CPython.
→ It prevents multiple native threads from executing Python code simultaneously, which can be a limitation for CPU-bound multi-threaded programs.

4 What is a decorator in Python ? Give an example.

→ A decorator is a function that takes another function as input and adds some extra functionality without modifying its code.

```
def my_decorator(func):
    def wrapper():
        print("Function is starting")
        func()
        print("Function is finished")
    return wrapper
```

```
@my_decorator
def say_hello():
    print("Hello Sagar")

say_hello()
# Function is starting
# Hello Sagar
# Function is finished
```

5 What are *args and **kwargs in Python ?

→ *args allows a function to accept any number of positional arguments.
→ **kwargs allows a function to accept any number of keyword arguments.

```
def demo(*args, **kwargs):
    print(args)   # tuple of positional arguments
    print(kwargs) # dictionary of keyword arguments
```

```
demo(1, 2, 3, name="Sagar", age=19)
# (1, 2, 3)
# {'name': 'Sagar', 'age': 19}
```

6 What is the difference between is and == in Python ?

→ == compares the values of two objects.
→ is checks if both variables refer to the same object (memory location).

```
a = [1, 2, 3]
b = [1, 2, 3]
print(a == b)    # True (same values)
print(a is b)    # False (different objects)
```

```
x = a
print(a is x)  # True (same object)
print(x == a)  # True
```

7 What is a list, tuple, set and dictionary comprehension ? Give an example.

```
# list comprehension
squares = [x*x for x in range(5)]
print(squares)  # [0, 1, 4, 9, 16]
```

```
# tuple comprehension
t = tuple(x*x for x in range(5))
print(t)    # (0, 1, 4, 9, 16]
```

```
# set comprehension
s = {x*x for x in range(5)}
print(s)   # {0, 1, 4, 9, 16]
```

```
# dict comprehension
d = {x: x*x for x in range(5)}
print(d)   # {0:0, 1:1, 2:4, 3:9, 4:16]
```

8 How to handle multiple exceptions in Python ?

We can handle multiple exceptions using multiple except blocks or a tuple of exceptions.

```
try:
    x = int(input("Enter a number: "))
    result = 10 / x
except (ValueError, ZeroDivisionError) as e:
    print("Error:", e)
```

```
# If user enters 0 → Error: division by zero
# If user enters non-number → Error: invalid literal for int()
# Works for both exceptions
```